Técnicas de Inteligência Artificial

PL

Licenciatura em Engenharia e Gestão de Sistemas de Informação

Filipe Portela

Exercícios

Exercício 2 - Resolução

```
% 2.1
- adiciona(X,L,[X|L]).
% 2.2
- apaga(X,[X|R],R).
- apaga(X,[Y|R1],[Y|R2]):- apaga(X,R1,R2).
% 2.3
- membro( X, [X|_] ).
- membro( X, [_|R] ) :- membro( X, R ).
% 2.4
- concatena([],L,L).
- concatena([X|L1],L2,[X|L3]):- concatena(L1,L2,L3).
% 2.5
- sublista(S,L):-
  concatena(_,L2,L), concatena(S,_,L2).
% 2.6
- comprimento(0,[]).
- comprimento(N,[_|R]):- comprimento(N1,R),
            N is 1 + N1.
% 2.7
- max(X,[X]).
- max(X,[Y|R]):- max(X,R), X > Y, !.
- %max(Y,[Y|R]):- max(X,R), X < Y, !.
- max(Y,[Y|_]).
% 2.8
- somatorio(0,[]).
- somatorio(X,[Y|R]):- somatorio(S,R), X is S+Y.
- media(X,L):-comprimento(N,L), somatorio(S,L),
            X is S/N.
% 2.9
- nelem(N,L,X):-
      nelem(N,1,L,X).
      nelem(N,N,[X|_],X):-!.
      nelem(N,I,[_|R],X):-
              I1 is I+1, nelem(N,I1,R,X).
```

```
% testar os predicados:
lista1([5,7,3,8,3,2]).

q1a(LR):-adiciona(1,[2,3],LR).
q1b(X):-adiciona(X,[2,3],[1,2,3]).
q1c(LR):-lista1(L),adiciona(1,L,LR).

q2a(LR):-apaga(a,[a,b,a,c],LR).
q2b(LR):-apaga(a,LR,[b,c]).
q2c(LR):- lista1(L), apaga(3,L,LR).
q2d(LR):- lista1(L), findall(L2,apaga(3,L,L2),LR).

q3a:-membro(b,[a,b,c]).
q3b(X):-membro(X,[a,b,c]).
q3c(LR):-findall(X,membro(X,[a,b,c]),LR).
q3d1:- lista1(L),membro(8,L).
q3d2:- lista1(L),membro(9,L).

q4a(LR):-concatena([1,2],[3,4],LR).
q4b(LR):-concatena([1,2],LR,[1,2,3,4]).
q4c(LR):-concatena(LR,[3,4],[1,2,3,4]).
q4d(LR):-lista1(L),concatena([9,9,9],L,LR).

q5a(X):-comprimento(X,[a,b,c]).
q5b(X):-lista1(L),comprimento(X,L).

q6a:- sublista([rui,ana],[rita,rui,ana,ivo]).
q6b(L):-sublista(L,[rita,rui,ana,ivo]).

q7a(X):-max(X,[3,2,1,7,4]).
q7b(X):-lista1(L),max(X,L).

q8(X):-media(X,[1,2,3,4,5]).

q9a:-nelem(2,[1,2,3],2).
q9b(X):-nelem(3,[1,2,3],X).
q9c(X):-nelem(4,[a,b,c,d,e,f,g],X).
```

Exercícios

Exercício 2 - Resolução

```
q1a(LR); q1b(X); q1c(LR); q2a(LR); q2b(LR); q2c(LR); q2d(LR); q3a; q3b(X); q3c(LR); q3d1; q3d2;
q4a(LR); q4b(LR); q4c(LR); q4d(LR); q5a(X); q5b(X); q6a; q6b(L); q7a(X); q7b(X); q8(X); q9a; q9b(X);
q9c(X)
LR = [1, 2, 3]
Next | 10 | 100 | 1,000 | Stop

?- q1a(LR); q1b(X); q1c(LR); q2a(LR); q2b(LR); q2c(LR); q2d(LR); q3a; q3b(X); q3c(LR); q3d1;
q3d2; q4a(LR); q4b(LR); q4c(LR); q4d(LR); q5a(X); q5b(X); q6a; q6b(L); q7a(X); q7b(X); q8(X);
q9a; q9b(X); q9c(X)
```

Program

```
:-dynamic(lista1/1).

% 2.1
adiciona(X,L,[X|L]).
% 2.2
apaga(X,[X|R],R).
apaga(X,[Y|R1],[Y|R2]):- apaga(X,R1,R2).
% 2.3
membro( X, [X|_] ).
membro( X, [_|R] ) :- membro( X, R ).
% 2.4
concatena([],L,L).
concatena([X|L1],L2,[X|L3]):- concatena(L1,L2,L3).
% 2.5
sublista(S,L):-
  concatena(_,L2,L), concatena(S,_,L2).
% 2.6
comprimento(0,[]).
comprimento(N,[_|R]):- comprimento(N1,R),
            N is 1 + N1.
% 2.7
max(X,[X]).
max(X,[Y|R]):- max(X,R), X > Y, !.
%max(Y,[Y|R]):- max(X,R), X < Y, !.
max(Y,[Y|_]).
% 2.8
somatorio(0,[]).
somatorio(X,[Y|R]):- somatorio(S,R), X is S+Y.
media(X,L):-comprimento(N,L), somatorio(S,L),
    X is S/N.
% 2.9
nelem(N,L,X):-
        nelem(N,1,L,X).
        nelem(N,N,[X|_],X):-!.
        nelem(N,I,[_|R],X):-
                I1 is I+1, nelem(N,I1,R,X).


% testar os predicados:
lista1([5,7,3,8,3,2]).
q1a(LR):-adiciona(1,[2,3],LR).
q1b(X):-adiciona(X,[2,3],[1,2,3]).
q1c(LR):-lista1(L),adiciona(1,L,LR).
q2a(LR):-apaga(a,[a,b,a,c],LR).
q2b(LR):-apaga(a,LR,[b,c]).
q2c(LR):- lista1(L), apaga(3,L,LR).
q2d(LR):- lista1(L), findall(L2,apaga(3,L,L2),LR).
q3a:-membro(b,[a,b,c]).
q3b(X):-membro(X,[a,b,c]).
q3c(LR):-findall(X,membro(X,[a,b,c]),LR).
q3d1:- lista1(L),membro(8,L).
q3d2:- lista1(L),membro(9,L).
q4a(LR):-concatena([1,2],[3,4],LR).
q4b(LR):-concatena([1,2],LR,[1,2,3,4]).
q4c(LR):-concatena(LR,[3,4],[1,2,3,4]).
q4d(LR):-lista1(L),concatena([9,9,9],L,LR).
q5a(X):-comprimento(X,[a,b,c]).
q5b(X):-lista1(L),comprimento(X,L).
q6a:- sublista([rui,ana],[rita,rui,ana,ivo]).
q6b(L):-sublista(L,[rita,rui,ana,ivo]).
q7a(X):-max(X,[3,2,1,7,4]).
q7b(X):-lista1(L),max(X,L).
q8(X):-media(X,[1,2,3,4,5]).
q9a:-nelem(2,[1,2,3],2).
q9b(X):-nelem(3,[1,2,3],X).
q9c(X):-nelem(4,[a,b,c,d,e,f,g],X).
```

```
q1a(LR); q1b(X); q1c(LR); q2a(LR); q2b(LR); q2c(LR); q2d(LR); q3a; q3b(X); q3c(LR); q3d1; q3d2; q4a(LR); q4b(LR); q4c(LR); q4d(LR); q5a(X); q5b(X); q6a; q6b(L);
q7a(X); q7b(X); q8(X); q9a; q9b(X); q9c(X)
LR = [1, 2, 3]
X = 1
LR = [1, 5, 7, 3, 8, 3, 2]
LR = [b, a, c]
LR = [a, b, c]
LR = [a, b, c]
LR = [b, a, c]
LR = [b, c, a]
LR = [5, 7, 8, 3, 2]
LR = [5, 7, 3, 8, 2]
LR = [[5, 7, 8, 3, 2], [5, 7, 3, 8, 2]]
true                                                    12
X = a
X = b
X = c
LR = [a, b, c]
true                                                    17
LR = [1, 2, 3, 4]
LR = [3, 4]
LR = [1, 2]
LR = [9, 9, 9, 5, 7, 3, 8, 3, 2]
X = 3
X = 6
true                                                    24
L = []
L = [rita]
L = [rita, rui]
L = [rita, rui, ana]
L = [rita, rui, ana, ivo]
L = []
L = [rui]
L = [rui, ana]
L = [rui, ana, ivo]
L = []
L = [ana]
L = [ana, ivo]
L = []
L = [ivo]
L = []
X = 7
X = 8
X = 3
true                                                    43
X = 3
X = d

?- q1a(LR); q1b(X); q1c(LR); q2a(LR); q2b(LR); q2c(LR); q2d(LR); q3a; q3b(X); q3c(LR); q3d1; q3d2; q4a(LR); q4b(LR); q4c(LR); q4d(LR); q5a(X);
q5b(X); q6a; q6b(L); q7a(X); q7b(X); q8(X); q9a; q9b(X); q9c(X)

Examples | History | Solutions        table results  Run!
```

Predicados Prolog

- **member**: O predicado member/2 é usado para verificar se um elemento pertence a uma lista. O member unifica o segundo argumento com um elemento da lista do primeiro argumento.
  - ?- member(2, [1,2,3]).
    - true.
- **append**: O predicado append/3 é usado para concatenar duas listas. O append unifica o terceiro argumento com a lista resultante da concatenação das duas listas do primeiro e segundo argumentos.
  - ?- append([1,2], [3,4], L).
    - L = [1, 2, 3, 4].
- **delete**: O predicado delete/3 é usado para remover todas as ocorrências de um elemento de uma lista, unificando o terceiro argumento com a lista resultante. O delete recebe como primeiro argumento o elemento a ser removido e como segundo argumento a lista original.
  - ?- delete(2, [1,2,3,2,4], Result).
    - Result = [1, 3, 4].
- **max_list**: O predicado max_list/2 é usado para encontrar o maior elemento em uma lista de números. O max_lista unifica o segundo argumento com o maior elemento da lista do primeiro argumento.
  - ?- max_list([1,3,2,4], Max).
    - Max = 4.

Predicados Prolog

- **nth1**: O predicado nth1 é usado para encontrar o elemento de uma lista em uma determinada posição. A indexação começa em 1. O nth0 é igual, mas a indexação começa no 0.
  - ?- nth1(2, [a,b,c], X).
    - X = b.
- **flatten**: O predicado flatten/2 é usado para transformar uma lista aninhada em uma lista simples.
  - ?- flatten([1,[2,3],[4,[5,6]]], L).
    - L = [1, 2, 3, 4, 5, 6].
- **length**: O predicado length/2 é usado para calcular o comprimento de uma lista. O length unifica o segundo argumento com o número de elementos na lista do primeiro argumento.
  - ?- length([1,2,3], L).
    - L = 3.
- **not** (\+): O predicado not/1 ou \+ é usado para negar uma regra. Este predicado unifica a regra com o false se não tiver soluções, falhando no caso contrário.
  - ?- not(member(5, [1,2,3])).
  - ?- \+ member(5, [1,2,3]).
    - true.

Predicados Prolog

- **findall**: O predicado findall/3 é usado para encontrar todas as soluções de uma regra e unificá-las em uma lista. É útil quando se precisa encontrar todas as soluções de uma regra e não apenas a primeira.
  - ?- findall(X, member(X, [1,2,3,2,4]), L).
    - L = [1, 2, 3, 2, 4].
- **select**: O predicado select/3 é usado para remover um elemento de uma lista e unificar o resultado com uma nova lista.
  - ?- select(2, [1,2,3], L).
    - L = [1, 3].
- **assert**: O predicado assert/1 é usado para adicionar factos e regras dinamicamente à base de conhecimento durante a execução do programa. asserta adiciona no início e assertz no final da lista.
  - ?- assert(pessoa(joao)).
    - true.
  - ?- pessoa(X).
    - X = joao.
- **retract**: O predicado retract/1 é usado para remover factos e regras da base de conhecimento em tempo de execução.
  - ?- assert(pessoa(joao)), retract(pessoa(joao)).
    - true.

Predicados Prolog

- **setof**: O predicado setof/3 é usado para encontrar todas as soluções de uma meta e retorná-las como uma lista ordenada e sem valores duplicados.
  - ?- setof(X, member(X, [1,2,3,2,4]), L).
    - L = [1, 2, 3, 4].
- **list_to_set**:O predicado list_to_set/2 é usado para converter uma lista em um conjunto, removendo todos os elementos duplicados. O list_to_set unifica o segundo argumento com uma lista contendo os mesmos elementos que a lista do primeiro argumento, mas sem duplicatas.
  - ?- list_to_set([1,2,3,2,4], Set).
    - Set = [1, 2, 3, 4].
- **Cut** (!): O predicado cut é usado para controlar e parar a procura de soluções em Prolog. O predicado é usado para parar a procura num determinado ponto, evitando que o Prolog explore outras alternativas.
  - ?- member(X, [1,2,3]), X = 2, !.
    - X = 2.
- **Flatten**: O predicado flatten/2 serve para achatar uma lista de listas, convertendo uma estrutura aninhada (com sublistas) numa única lista contendo todos os elementos não-lista em ordem, da esquerda para a direita
  - ?- flatten([a, [b, c], [[d]], e], Lista).
    - Lista = [a, b, c, d, e].

Predicados Prolog

- **consult ou []:** O predicado consult/1 ou [] é usado para incluir um arquivo de Prolog na base de conhecimento atual.
  - ?- consult('meu_arquivo.pl').
  - ?- [meu_arquivo].
- **modules**: Os módulos permitem que o programador separe o código em unidades lógicas e independentes, evitando conflitos de nomes e melhorando a modularidade do código. Os módulos são definidos usando a diretiva module/2 e podem ser carregados usando a diretiva use_module/1.
  - :- module(meu_modulo, [meu_predicado/1]).
    - meu_predicado(X) :- ...
  - ?- use_module(meu_modulo).
- **sort**: O predicado sort/2 é usado para ordenar uma lista. O sort retorna uma nova lista contendo os mesmos elementos que a lista original, mas em ordem crescente.
  - ?- sort([3,2,1,4], SortedList).
    - SortedList = [1,2,3,4].
- **keysort**: O predicado keysort/2 é utilizado para ordenar uma lista de pares chave-valor com base na chave. O predicado retorna uma nova lista contendo os mesmos pares, mas em ordem crescente com base na chave:
  - ?- keysort([b-2, a-1, c-3], SortedPairs).
    - SortedPairs = [a-1, b-2, c-3].

Predicados Prolog

Before:

```
colaborador(rita).
colaborador(joao).
```

asserta(colaborador(**vitor**)) →

After:

```
colaborador(vitor).
colaborador(rita).
colaborador(joao).
```

assert(colaborador(**rui**)) →

After:

```
colaborador(rita).
colaborador(joao).
colaborador(rui).
```

assertz(colaborador(**miguel**)) →

After:

```
colaborador(rita).
colaborador(joao).
colaborador(miguel).
```

Exemplos

- /* Animals Database */
  animal(mammal,tiger,carnivore,stripes).
  animal(mammal,hyena,carnivore,ugly).
  animal(mammal,lion,carnivore,mane).
  animal(mammal,zebra,herbivore,stripes).
  animal(bird,eagle,carnivore,large).
  animal(bird,sparrow,scavenger,small).
  animal(reptile,snake,carnivore,long).
  animal(reptile,lizard,scavenger,small).

- **?- animal(mammal,X,_,_).**

  animal(mammal,X,_,_).

  X = tiger
  X = hyena
  X = lion
  X = zebra

- **?- animal(reptile,R,_,mane).**

  animal(reptile,R,_,mane).

  false

- /* Dating Agency Database */
  person(bill,male).
  person(george,male).
  person(alfred,male).
  person(carol,female).
  person(margaret,female).
  person(jane,female).

- couple(X,Y) :- person(X,male),person(Y,female).

  couple(X,Y).

  X = bill,
  Y = carol
  X = bill,
  Y = margaret
  X = bill,
  Y = jane
  X = george,
  Y = carol
  X = george,
  Y = margaret
  X = george,
  Y = jane
  X = alfred,
  Y = carol
  X = alfred,
  Y = margaret
  X = alfred,
  Y = jane

Exemplos

Code:

```
content(merry, red).
content(mia, red).
content(henry, red).
content(Names, red) :- colors(Names).
colors(mia).
content(mia, yellow).
content(henry, yellow).
content(Names, yellow) :- colors(Names).
colors(mia).
```

In the console shown below.

Code:

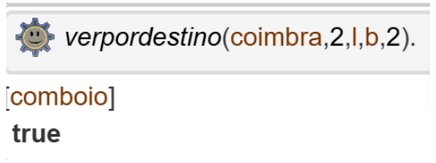
```
| ?- findall(A, content(A, red), Result).
```

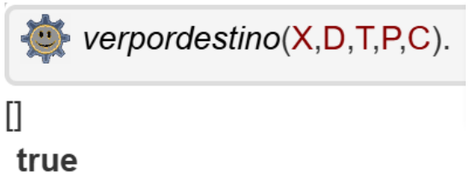
Output:

Exemplos

- /*destino(destino,transporte,duracao(min),duracao(letra),preco(euro),preco(letra),conforto,objetivo de viagem,deseja conduzir(1 sim,2 nao))*/
- destino(braga, carro, 25, r,5.36,c, 3, [historico,gastronomia],1).
- destino(braga, autocarro, 25,r, 6,c, 3, [historico,gastronomia],2).
- destino(braga, comboio, 66,m, 3.55,b, 3, [historico,gastronomia],2).
- destino(porto, carro, 45,r, 9.83,c, 3, [compras,romantico],1).
- destino(porto, autocarro, 60,m, 6,b, 2, [compras,romantico],2).
- destino(porto, comboio, 67,m, 3.25,b, 3, [compras,romantico],2).
- destino(coimbra, carro, 113,r, 29.32,c, 3, [cultura,romantico],1).
- destino(coimbra, autocarro, 145,m, 14.5,c, 2, [cultura,romantico],2).
- destino(coimbra, comboio, 231,l, 11.4,b, 2, [cultura,romantico],2).

- **verpordestino(X,D,T,P,C):-** findall(Y,(destino(X,Y,_,T,_,P,CC,_,DD), DD == D, CC == C), K), print(K).
- **?- verpordestino(porto,1,r,c,3).**
- **?- verpordestino(coimbra,2,l,b,2).**
- **?- verpordestino(X,D,T,P,C).**





Exemplos

```
makefoods :-
    assert(food(apple)),
    assert(food(broccoli)),
    assert(food(carrot)),
    assert(food(lettuce)),
    assert(food(rice)).
```

Evaluating `makefoods` adds facts to the database:

```
?- food(X).
ERROR: Undefined procedure: food/1

?- makefoods.
Yes

?- food(X).
X = apple
```

A food can be "removed" with `retract`:

```
?- retract(food(carrot)).
Yes

?- food(carrot).
No
```

```
makefoods :-
    retractall(food(_)),
    assert(food(apple)),
    assert(food(broccoli)),
    assert(food(carrot)),
    assert(food(lettuce)),
    assert(food(rice)).
```

Without `retractall`, each call to `makefoods` creates an identical, duplicated set of foods.

Here's the behavior of the first version of `makefoods`:

```
?- makefoods, makefoods, makefoods.
Yes

?- findall(F, food(F), Foods).
Foods = [apple, broccoli, carrot, lettuce, rice, apple,
broccoli, carrot, lettuce|...]
```

Exercícios

Exercício 3

Considere a seguinte BD sobre clientes de stands de automóveis:

Stand	Nome	Nº Cliente	Idade	Profissão	Compras
Vegas	Rui	2324	23	Médico	Carro Audi A2 por 20000 euros Carro BMW Serie3 por 30000 euros
Vegas	Rita	2325	32	Advogado	Carro Audi A3 por 30000 euros
Vegas	João	2326	26	Professor	Moto Honda GL1800 por 26000 eur.
Vegas	Ana	2327	49	Médico	Carro Audi A4 por 40000 euros Carro BMW Serie3 por 32000 euros Carro Ford Focus por 24000 euros
Miami	Rui	3333	33	Operário	Carro Fiat Panda por 12000 euros
Miami	Paulo	3334	22	Advogado	Carro Audi A4 por 36000 euros
Miami	Pedro	3335	46	Advogado	Carro Honda Accord por 32000 eur. Carro Audi A2 por 20000 euros

1. Registe em prolog todos os dados relevantes da BD, utilizando factos com a notação:
   - **stand**(nome_stand,LC).
     - onde LC é uma lista de clientes do tipo: [cliente(nome,num,id,prof,C1),cliente(nome2,num2,id2,prof2,C2),…]
     - onde C1, C2 são listas de compras do tipo: [carro(marca1,modelo1,preco1), moto(marca2,modelo2,preco2),…]
2. Defina em Prolog os seguintes predicados:
   - **a)** **listar_clientes**(X,LC) – devolve a lista LC com o nome de todos clientes do stand X;
   - **b)** **listar_dados**(X,C,D) – devolve a lista D com todos dados (i.e.: numero, idade e profissão) do cliente com o nome C do stand X;
   - **c)** **listar_carros**(X,LM) – devolve a lista LM com o nome de todas as marcas de carros vendidos pelo stand X.
   - **d)** **listar_advogados**(LA):- devolve a lista LA com o nome de todos os advogados de todos os stands;
   - **e)** **preco_medio**(X,Med) - devolve o preço médio (Med) de todos os carros vendidos por um stand.
     - Nota: pode re-utilizar o predicado media(X,L) do exercício anterior;
   - **f)** **altera_id**(X,C,Id) – altera a idade do cliente C do stand X para Id. Nota: deve usar os predicados do Prolog assert e retract.

Notas: Assuma que existem já definidos os seguintes predicados SWI-prolog:

- **flatten**(L1,L2) – remove todos os [] extra de L1, devolvendo o resultado em L2;
- **list_to_set**_(L1,L2) – remove elementos repetidos de L1, devolvendo L2;

Por exemplo, flatten([[1],[2,3]],[1,2,3]) e list_to_set([1,2,2,3],[1,2,3]) dão verdade.

Exercícios

Exercício 3

```
% ------- base de dados --------
:-dynamic(stand/2),

stand(vegas,[

                cliente(rui,2324,23,medico,[carro(audi,a2,20000),carro(bmw,serie3,30000)]),

                cliente(rita,2325,32,advogado,[carro(audi,a3,30000)]),

                cliente(joao,2326,26,professor,[moto(honda,gl1800,26000)]),

                cliente(ana,2327,49,medico,[carro(audi,a4,40000),

                                                carro(bmw,serie3,32000),carro(ford,focus,24000)])

            ]).
```

- % - devolve uma lista com o nome de todos os clientes de um dado stand

```
listar_clientes(X,LC):- stand(X,L),

                        findall(C,membro(cliente(C,_,_,_,_),L),LC).
```

listar_clientes(X,LC)

LC = [rui, rita, joao, ana],
X = vegas

```
% - devolve os dados de cliente (todos excepto o nome), atraves do nome do cliente e nome do
stand

listar_dados(X,C,D):- stand(X,L),

                      findall((N,ID,P),membro(cliente(C,N,ID,P,_),L),D).
```

listar_dados(X,C,D)

D = [(2324,23,medico), (2325,32,advogado), (2326,26,professor), (2327,49,medico)],
X = vegas

Exercícios

Exercício 3

Considere a seguinte BD sobre clientes de stands de automóveis:

Stand	Nome	Nº Cliente	Idade	Profissão	Compras
Vegas	Rui	2324	23	Médico	Carro Audi A2 por 20000 euros Carro BMW Serie3 por 30000 euros
Vegas	Rita	2325	32	Advogado	Carro Audi A3 por 30000 euros
Vegas	João	2326	26	Professor	Moto Honda GL1800 por 26000 eur.
Vegas	Ana	2327	49	Médico	Carro Audi A4 por 40000 euros Carro BMW Serie3 por 32000 euros Carro Ford Focus por 24000 euros
Miami	Rui	3333	33	Operário	Carro Fiat Panda por 12000 euros
Miami	Paulo	3334	22	Advogado	Carro Audi A4 por 36000 euros
Miami	Pedro	3335	46	Advogado	Carro Honda Accord por 32000 eur. Carro Audi A2 por 20000 euros

1. Registe em prolog todos os dados relevantes da BD, utilizando factos com a notação:
   - **stand**(nome_stand,LC).
     - onde LC é uma lista de clientes do tipo: [cliente(nome,num,id,prof,C1),cliente(nome2,num2,id2,prof2,C2),…]
     - onde C1, C2 são listas de compras do tipo: [carro(marca1,modelo1,preco1), moto(marca2,modelo2,preco2),…]
2. Defina em Prolog os seguintes predicados:
   - **a)** **listar_clientes**(X,LC) – devolve a lista LC com o nome de todos clientes do stand X;
   - **b)** **listar_dados**(X,C,D) – devolve a lista D com todos dados (i.e.: numero, idade e profissão) do cliente com o nome C do stand X;
   - **c)** **listar_carros**(X,LM) – devolve a lista LM com o nome de todas as marcas de carros vendidos pelo stand X.
   - **d)** **listar_advogados**(LA):- devolve a lista LA com o nome de todos os advogados de todos os stands;
   - **e)** **preco_medio**(X,Med) - devolve o preço médio (Med) de todos os carros vendidos por um stand.
     - Nota: pode re-utilizar o predicado media(X,L) do exercício anterior;
   - **f)** **altera_id**(X,C,Id) – altera a idade do cliente C do stand X para Id. Nota: deve usar os predicados do Prolog assert e retract.

Notas: Assuma que existem já definidos os seguintes predicados SWI-prolog:

- **flatten**(L1,L2) – remove todos os [] extra de L1, devolvendo o resultado em L2;
- **list_to_set**_(L1,L2) – remove elementos repetidos de L1, devolvendo L2;

Por exemplo, flatten([[1],[2,3]],[1,2,3]) e list_to_set([1,2,2,3],[1,2,3]) dão verdade.

Resolve o exercício com a ajuda do GEM